

Домашка 02 — Agile Manifesto vs. мій робочий процес

Автор: Андрій Пономарьов / Claude Code Opus 4.8 **Завдання:** прочитати Agile Manifesto, стисло описати власний робочий процес, показати відмінності від «класичного» agile, запропонувати покращення. Акцент — на організації роботи в команді.

1. Що каже Agile Manifesto (коротко)

4 цінності (ліве важливіше за праве):

1. Люди та взаємодія — понад процеси й інструменти
2. Працюючий продукт — понад вичерпну документацію
3. Співпраця із замовником — понад узгодження умов контракту
4. Готовність до змін — понад слідування плану

12 принципів зводяться до кількох ідей: рання й часта поставка цінності, вітати зміни навіть пізно, короткі ітерації, щоденна спільна робота бізнесу й розробників, самоорганізовані команди, технічна досконалість, простота («максимізувати обсяг невиконаної роботи») і регулярна рефлексія для покращення.

DevOps (рекомендована стаття Atlassian + домашка 01 про CALMS) — це продовження agile: agile прибрав бар'єри *всередині* команди розробки, DevOps прибирає бар'єр між **dev** і **ops** через автоматизацію, CI/CD і культуру спільної відповідальності. Agile Manifesto (2001) написаний ще до CI/CD — тому «класичний agile» сам по собі CI/CD не вимагає.

2. Чому еволюціонував SDLC — особистий погляд

На мою думку, еволюція підходів до розробки програмного забезпечення — це не просто зміна моди чи інструментів, а природне відображення темпу сучасного світу.

Якщо подивитися історично, ще в 60-х роках людина могла закінчити університет, отримати професію і працювати в ній десятки років майже без фундаментальної перекваліфікації. Наприклад, інженери атомної енергетики, залізниці чи телекомунікацій. Базових знань вистачало на всю кар'єру.

Сьогодні це вже не так. Швидкість технологічних змін настільки висока, що знання мають дуже короткий життєвий цикл. Я бачу це на власному прикладі. Коли я прийшов у професію у 2000 році, банкомати були новою, складною і перспективною технологією. Про cash recycling, deposit automation або advanced self-service майже ніхто не говорив як про реальність. Сьогодні, через 26 років, банкоматна індустрія для багатьох виглядає як legacy, а основний вектор розвитку змістився у сторону cashless, API-based payments та digital-first сервісів.

При цьому самі банкомати теж змінилися: від закритих монолітних систем до платформ із touchscreen, API integrations, online services та CI/CD pipelines.

На мою думку, саме ця швидкість змін і стала причиною еволюції SDLC.

Класичний **Waterfall** добре працював у світі монолітних систем, де архітектура була цілісною, зміни — дорогими, а цикл розробки — довгим. І навіть зараз для великих монолітів або highly regulated environments Waterfall залишається актуальним. Але проблема в тому, що якщо продукт розробляється 12–18 місяців, то на момент delivery вимоги ринку вже можуть кардинально змінитися.

Саме тому з'явився **Agile** — він дозволив скоротити цикл delivery, працювати ітеративно та швидше адаптуватися до змін. Але для цього сама архітектура теж мала змінитися. Так з'явилися **мікросервіси**: вони дозволяють змінювати окремі частини системи незалежно. Якщо замовнику завтра потрібна нова кнопка зі звітом, це вже не означає перебудову всього продукту. Але мікросервіси породили нові виклики: API communication, synchronous vs asynchronous processing, message queues, service discovery, fault tolerance.

Далі з'явилася проблема середовища: «у мене працює, у тебе не працює». Відповіддю стала **контейнеризація** (Docker, Podman, LXC) — контейнери стандартизували запуск ПЗ і зробили deployment передбачуваним.

Наступний великий крок — **Git**. Він дозволив організувати колективну розробку так, щоб десятки людей могли одночасно працювати над одним продуктом без хаосу. А платформи типу GitHub та GitLab побудували навколо цього екосистему collaboration.

Але код у Git — це ще не продукт. Його потрібно перевірити, протестувати, зібрати, доставити, запустити і перевірити після запуску. Саме тут з'являється **CI/CD** — автоматизований шлях від source code до production:

```
код → GitHub → GitHub Actions (linting, tests, secret scanning, security scanning)
    → збірка container image → push у registry
    → деплой на VPS / Kubernetes → health checks
```

Так delivery стає швидким, передбачуваним і безпечним. У сучасному світі майже все переходить у модель «**as a code**»: Infrastructure as Code, Policy as Code, Documentation as Code, Security as Code — а CI/CD стає універсальним механізмом оркестрації всіх цих процесів.

І зараз ми бачимо наступну революцію — **Artificial Intelligence**. Штучний інтелект радикально прискорює створення рішень. Те, що ще п'ять років тому вимагало команди з кількох людей і місяців роботи, сьогодні одна людина може зібрати за одну ніч і вранці запустити в production. На мою думку, AI — це не просто новий інструмент. Це наступний логічний етап еволюції DevOps, де швидкість переходить на зовсім інший рівень.

Саме на цьому тлі й варто читати мої три кейси нижче: банк (C) — це світ монолітів і Waterfall, з якого все починалось; інструмент збору візиток (B) — це вже мікросервіс + контейнери + CI/CD; а робота з Claude Code (A) — це той самий «наступний етап», де AI стискає цикл до годин.

3. Мій поточний робочий процес (3 реальні кейси)

Кейс А — робота з Claude Code (цей курс)

Цикл максимально короткий: **я ставлю задачу природною мовою** → **агент виконує** → **я рев'юю результат** → **коригую**. Немає спринтів, естимейтів, беклогу й церемоній — це безперервний потік (ближче до Kanban/DevOps, ніж до Scrum). «Команда» = одна людина (я — у ролі product owner + reviewer) плюс AI-агент (у ролі розробника). Комунікація асинхронна, текстова, але зворотний зв'язок миттєвий.

Кейс В — інструмент збору візиток і лідів

Шлях: від розмитої ідеї «**а давайте якось зберігати візитки та ліди**» → до робочої архітектури за лічені ітерації:

```
Telegram (джерело: фото візитки / повідомлення)
  ↓
GCP (бекенд: обробка, парсинг, логіка)
  ↓
ClickUp (цільова система: лід як картка)
```

Тулчейн: **Claude** → **Git** → **GitHub** → **GitHub Actions** — тобто повноцінний CI/CD. Тестування йде *в процесі*, а не окремою фазою наприкінці. Архітектура не була спроектована «згори донизу» — вона **проросла** з потреби (емерджентний дизайн, принципи 10–11 маніфесту).

Кейс С — як це робилося в банку ~20 років тому

Класичний waterfall, послідовні фази без перетину:

Збір вимог → Скоупінг → Архітектура → Планування ресурсів → Розробка → Тестування → UAT → Pilot → Rollout

CI/CD **немає**. Реліз — рідкісна, велика, ризикована подія. Зворотний зв'язок від замовника по суті лише на UAT/pilot — у самому кінці, коли зміна вже дорога. Команда розбита на «силови» за фазами (аналітики → архітектори → розробники → QA → ops), кожен «перекидає» роботу далі.

4. Відмінності від «класичного» agile

Аспект	Кейс С (банк, waterfall)	Класичний agile (Scrum)	Кейси А/В (мій сьогоднішній)
Поставка	рідко, наприкінці	кожні 1–4 тижні (спринт)	безперервно, по готовності
Зворотний зв'язок	лише UAT/pilot	демо в кінці спринта	у межах хвилин (рев'ю одразу)
Зміни	дорогі, через change request	вітаються між спринтами	вітаються будь-коли, в потоці
Документація	вичерпна, наперед	помірна	мінімальна, код = джерело правди
Архітектура	велике проєктування наперед	потроху	емерджентна (В)
Команда	силови за фазами	крос-функційна, самоорганізована	людина + AI-агент
CI/CD	немає	не обов'язковий (поза маніфестом)	базовий від першого дня (В)

Ключове спостереження. Банк (С) суперечить майже всім цінностям маніфесту: процес понад людей, документація понад продукт, план понад зміни. Мій сьогоднішній процес (А/В) реалізує цінності agile й одночасно **виходить за межі маніфесту 2001 року** в бік DevOps: CI/CD, автоматизоване тестування, один безперервний потік замість ітерацій-«коробочок».

Цікавий нюанс щодо **командної організації**: agile передбачає *самоорганізовану крос-функційну команду людей* (принципи 4, 6, 11) і «спілкування віч-на-віч» як найефективніше. У

моєму випадку команда переозначена — **1 людина + AI**. «Віч-на-віч» замінено на асинхронний текст, але затримка зворотного зв'язку близька до нуля, тож дух принципу (швидка взаємодія) збережено, навіть якщо буква (face-to-face) — ні.

5. Що можна покращити

1. **Definition of Done як специфікація (spec-driven).** Перед стартом задачі фіксувати критерії приймання — не вільним текстом, а як специфікацію, якій агент слідує (spec-driven development, напр. OpenSpec / подібні інструменти). Зменшує переробку: зараз розбіжності часто спливають уже під час рев'ю.
 2. **Регулярна рефлексія (принцип 12).** Зараз ретро — стихійне. Варто раз на N задач коротко фіксувати «що сповільнювало» і правити CLAUDE.md / пам'ять агента.
 3. **Регулярні рев'ю — зокрема за участі інших агентів.** Не покладатися лише на рев'ю людиною: підключати окремого агента-рев'юера (а краще кількох, незалежно), який перевіряє роботу основного. Це multi-agent / adversarial review — ловить помилки, яких один виконавець не бачить.
 4. **Готовність до зростання команди й зсув до архітектурної експертизи.** З ростом проєкту додавати учасників (людей чи агентів) і готуватися до цього заздалегідь: branch protection, обов'язкове code review у GitHub Actions, спільні конвенції (зараз процес «заточений» під одного оператора). Паралельно змінюється цінність навичок: **системна / архітектурна експертиза стає важливішою за предметні знання напам'ять**. Приклад: не обов'язково пам'ятати всі linux-команди для налаштування роутингу — але обов'язково розуміти стек TCP/IP загалом.
-

6. Висновок

Класичний agile витіснив waterfall-модель банку (кейс C), прибравши силоси й наблизивши зворотний зв'язок. Мій сьогоднішній процес з AI (кейси A/B) іде ще на крок далі: він **бере цінності agile й додає DevOps-автоматизацію**, а разом переозначає саме поняття «команда» — від десятка спеціалістів за фазами до зв'язки «людина-замовник/рев'юер + AI-розробник» із майже миттєвим циклом. Напрямок покращень — не «більше процесу», а чіткіші критерії готовності (spec-driven), незалежні рев'ю (зокрема іншими агентами) і готовність до зростання команди, де **системна / архітектурна експертиза важить більше за предметні знання напам'ять**.